Python operators :-

→ operator is syntax that tells python to perform an operation on objects

operators :- operators are special symbols like, +, -, * etc
operands :- value on which the operator is applied

Arthematic operators :-

→ Arthematic operators is used in bin operands

→ + → addition → x+y
→ - → Subtraction → x-y
→ * → mutiplicatn → x*y
→ / → Divisin → x/y
→ % → modulus → x%y
→ ** → exponent → x**y
→ // → floor divisn → x//y

Assignment operator :-

→ it is used to assign values for varibles.
→ update them using different operations
→ this operator is mainly assign result to an expression to a variable.

code :-

```
a = 3      → assignment operator
b = 5
c = a + b
print(cc)   #8
```

→ Addition assignment →

```
a = 5, b = 5;
a += b
print(a) # 8
```

→ like wise we can do for the all arthemitic operators and bitwise operators

AND

inputs		outputs
A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

OR

inputs		Output
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

X-OR i/p -> o/p

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

Not

input	O/P
0	1
1	0

→ Bitwise assignment example

```
a = 5
b = 3
a &= b
print(a)  ⟹
```

```
 8 4 2 1
 0 1 0 1
 0 0 1 1
---------
 0 0 0 1  → (1)
---------
```

→ This are called Compand assignment operator

instead

$x = x + 5$

you can write

$x += 5$ → Compand assignment operator.

→ walrus operator := (:=) → (assign a value in expression and use the same)

→ if x := 10:
    print(x)
} invalid

if (x := 10):
    print(x)
} Because of it is (walrus operator)

#106

Comparison operators :-

→ it is also called relational operators.
→ Comparisim operators are used to compared values.
→ it returns true or false according to the condition

Comparsion operator are

operator	meaning	Example	Result
==	equal	10 == 10	True
!=	Not equal	10 != 10	False
>	greater than	20 > 10	True
<	Less than	10 < 20	True
>=	greater/equal	10 >= 10	True
<=	Lessthan/equal	10 <= 10	True

→ Python allows you to chain comparison operators.

```
x=5
print(1<x<5)  #True
print(1<x and x<10)  #true
```

Logical operators :-

→ Logical operators are used to combine or modify conditional and return the boolean value (True/false)
→ They are commonly used in conditional statement to control flow of program based on logical condition

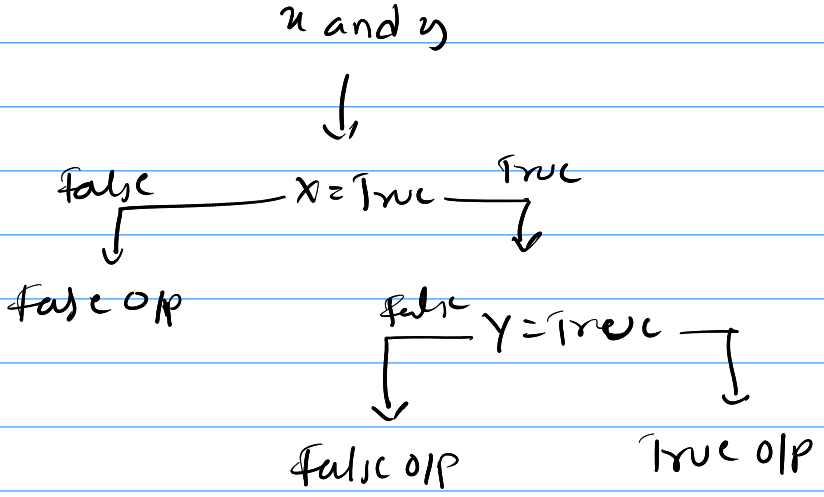
① And ② or ③ Not

x	y	x and y	x or y	Not(x)	Not(y)
T	T	T	T	F	F
T	F	F	T	F	T
F	T	F	T	T	F
F	F	F	F	T	T

AND operator :-

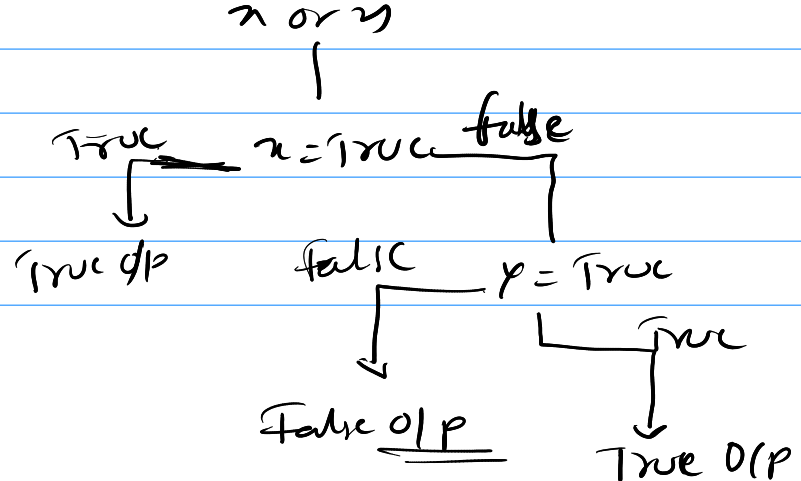
→ AND return True

→ Both the operands are True Use it return false

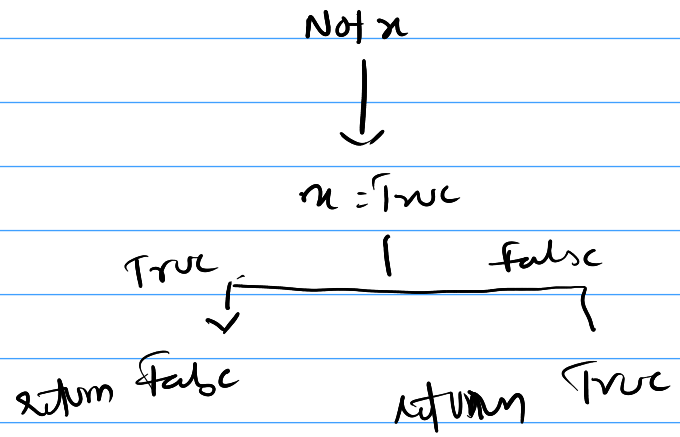


Or operator :-

→ boolean operator returns True if either of operands True



Not operator :-



Bitwise operators :-

→ Bitwise operators అనేవి bit wise calculation చేయడానికి integers ని bits గా మారుస్తాయి

→ it convert the integer to Binary format. Then operations performed on each bit or corresponding pair bits

→ it return the Decimal.

Bitwise And (&) :-

		8 4 2 1
5 & 3	5 →	0 1 0 1
	3 →	0 0 1 1
		0 0 0 1 → (1)

Truth table

0 & 0 = 0
0 & 1 = 0
1 & 0 = 0
1 & 1 = 1

Bitwise OR (|) :-

5 | 3

0 1 0 1 → 5
0 0 1 1 → 3
0 1 1 1 → 7

Truth table

0 | 0 = 0
0 | 1 = 1
1 | 0 = 1
1 | 1 = 1

identity operator:-

① is
② is not

→ This is to check whether two objects belongs to same memory location

```
a = 10
b = 10
print(a is b)   # True
```

```
a = 10
b = 10
print(a is not b)   # False
```

Membership operator:-

→ it is used to the is this value present in the collection
→ it return True False
'in':- checks whether a value exist inside something
not 'in':- checks whether a value does not exist inside something

```
a = [20, 30, 40]
print(20 in a)      # True
print(20 not in a)  # False
```

→ in Dictionary it only check the key

```
a = {"name": "guru", "age": 21}
print("name" in a)   # True
print("guru" in a)   # False
```

Bitwise XOR (^) :-

Truth table

0 ^ 0 = 0

0 ^ 1 = 1

1 ^ 0 = 1

1 ^ 1 = 0

5 ^ 3

	8	4	2	1
5	0	1	0	1
3	0	0	1	1
	0	1	1	0 → 6

Bitwise NOT (~) :-

→ it works only on on number

0 → 1

1 → 0

→ 8 Bits

5 → 00000101 → flip → 11111010

11111010 + 1 (add)

00000110 = 6

0 → positive

1 → negative

*imp relationship

~x = -(x+1)

~5 = -(5+1)

= -6

= -6

Left shift (<<)

→ Left shift move bits towards the left

Ex :- 5 << 1

0000 0101 → 5

0000 1010 → 10

→ we need multiplate 2^{power}

5 << 1 ⇒ 5 x 2^1 = 10

x << n ⇒ x x 2^n = Ans

Right shift :- (>>)

→ The Right shift moves bits towards right

```
                    16 8 4 2 1
20>>2               20 :- 0 0 1 0 1 0 0
                          0 0 0 1 0 1 0
n>>n = n/2^n              0 0 0 1 0 1 0
```

(5)

$20 >> 2 = 20/2^2$

$= 20/4$

$= 5$

Ternary operator :- (also called conditional expression)

→ it assign a value if condition True or, and another if it is false

n = 6

n = "positive" if n>1 else "negative"

print(n) → positive

n = "positive" if n>1 else "negative" if n<0 else "zero"

print(n)

→ operator precedence :-

→ which one have highest priority to run or check